

Preferendus-IMAP-IGS

Preference-Based Optimisation — IMAP Inter-Generational Solver (IGS) Framework

About

This is the home of the IMAP Inter-Generational Solver (IGS) framework - a collection of preference-based optimisation solvers built on the IMAP (Integrative Maximisation of Aggregated Preferences) aggregator. The page hosts four repositories, each a distinct IGS instantiation targeting different problem classes: IGS-GA-I, IGS-GA-II, IGS-BRKGA, and IGS-SI.

The open-source Preferendus software for preference-based optimisation used for the IMAP demonstrators in the ODESYS book [1] is publicly available via the GitHub repository: <https://github.com/TUdelft-Odesys>, including the early IMAP/IGS 1.0 implementation.

Repositories

IGS-GA-I

<https://github.com/TUdelft-IMAP-IGS/IGS-GA-I>

The original IGS-GA algorithm — the foundational Inter-Generational Solver using a Genetic Algorithm backbone. This is a user-friendly implementation, equivalent to the original version linked to all ODESYS book examples. It integrates IMAP preference aggregation directly into standard GA survival selection.

IGS-GA-II

<https://github.com/TUdelft-IMAP-IGS/IGS-GA-II>

An improvement on IGS-GA-I, built on the NSGA-II genetic loop (crossover, mutation, mating) but replacing non-dominated sorting and crowding distance with IMAP affine aggregation survival. Designed for benchmarking against reference test suites (e.g. DAS-CMOP). Supports multi-stakeholder preference models, constraint handling via the Constraint Dominance Principle (CDP), tournament and truncation selection, and an optional cross-generation archive to stabilise IMAP Z-score normalisation.

IGS-BRKGA

<https://github.com/TUdelft-IMAP-IGS/IGS-BRKGA>

A Biased Random-Key Genetic Algorithm (BRKGA) instantiation of the IMAP-IGS framework, developed for combinatorial and constraint-heavy optimisation problems. Used as the solver for the Heterogeneous Vessel Allocation and Scheduling Problem (HVASP), and the Multi-Objective Vehicle Routing Problem with Time Windows (MO-VRPTW).

IGS-SI

<https://github.com/TUdelft-IMAP-IGS/IGS-SI>

A Swarm Intelligence (SI) instantiation of the IMAP-IGS framework, targeting graph-based and routing optimisation problems. Implements the Preferendus method via Ant Colony Optimisation (ACO), with the same preference-aggregation interface as the GA-based solvers. Applicable to directed graph problems such as route planning and Markov-chain modelling.

IMAP-IGS Terminology

IGS-GA-II: A GA which inherits the genetic loop (crossover, mutation, mating) from NSGA-II but replaces the non-dominated sorting and crowding distance with IMAP affine aggregation survival.

IMAP (Integrative Maximisation of Aggregated Preferences):

The preference function $P_{k,i}$ maps an actor-independent performance function vector F to a corresponding preference score, reflecting the goal-oriented evaluation of actor k , such that $P_{k,i}(F) \in [0, 100]$, where $P_{k,i}$ acts component-wise on the i^{th} element of F . The relatively worst (least preferred) and best (most preferred) performance levels are assigned preference scores of 0 and 100, respectively, representing an arbitrary reference scaling since preferences possess only relative meaning within a one-dimensional affine space, i.e., all preference values are defined only up to relative differences and not in absolute terms [Wolfert (2026)].

Open Design-Decision System (ODESYS)

Odesys operator: let a complex, multifaceted socio-physical problem comprising performance functions indexed by i and actor preference functions indexed by k , and defined over the endogenous decision vector x , the exogenous parameter vector y , be formulated as:

$$\text{OD} (P_{k,i}(F(x, y)); w'_{k,i})$$

Preference functions: Preference is a subjective and contextual construct that is not an absolute physical property but represents an individual's relative choices, thereby defining the decision space. The preference function $P_{k,i}$ maps an actor-independent performance function vector F to a corresponding preference score, reflecting the goal-oriented evaluation of actor k , such that $P_{k,i}(F) \in [0, 100]$, where $P_{k,i}$ acts component-wise on the i^{th} element of F . The relatively worst (least preferred) and best (most preferred) performance levels are assigned preference scores of 0 and 100, respectively, representing an arbitrary reference scaling since preferences possess only relative meaning within a one-dimensional affine space, i.e., all preference values are defined only up to relative differences and not in absolute terms [4].

Performance function: Here, $F = (f_1, \dots, f_I)$ denotes the vector of system performance functions, where each $f_i(x, y, t)$ denotes the i^{th} performance function, and I is the total number of performance functions. These functions describe the physical behaviour of the system, i.e. its capabilities, representing what the system *can* perform. The vector x represents the 'controllable' design-decision vector, comprising both product and process variables that can be deliberately selected and adjusted by the decision maker. The vector y is the 'uncontrollable' parameter vector, containing contextual variables, including external product and process conditions that influence system behaviour but cannot be controlled.

Actors/stakeholders: Actors may express the relative importance of performance dimensions through preference weights. This accounts for both (i) the actor's overall importance in the decision process, represented by the global weight w_k (with $w_k = \frac{1}{K}$ in the case of equivalent stakeholder importance), and (ii) the local importance weight $w_{k,i}$ assigned by stakeholder k to performance function i , such that the combined weight for each preference function is given by $w'_{k,i} = w_k \cdot w_{k,i}$ with $\sum_k w_k = 1$, $\sum_i w_{k,i} = 1 \forall k$, and consequently $\sum_{k,i} w'_{k,i} = 1$. When an actor has no interest in a particular performance function, the corresponding local weight is set to zero.

A-fine-aggregator

A-fine-aggregator: A is the a-fine-aggregator, a linear weighted centroid operator that aggregates actors' normalised preference scores over their associated performance evaluations. The operator acts on the z -normalised preference scores $z_{k,i}(x)$, weighted by $w'_{k,i}$, inducing a preference-consistent aggregated ranking over the design-decision space, see [4] for details on unique preference aggregation using Preference Function Modelling (PFM)).

Best-fit-for-common-purpose solution is defined as the unique design-decision vector $\mathbf{x}^*$ that maximises the aggregated preference operator:

$$\mathbf{x}^* = \operatorname{argmax}_{\mathbf{x} \in \mathcal{S}_{f,a}} Z, \quad Z = \sum_{k,i} w'_{k,i} Z_{k,i}(\mathbf{x}), \quad \sum_{k,i} w'_{k,i} = 1$$

Core aggregation function. Steps:

1. Transpose $\mathbf{p}$ so rows = individuals, columns = criteria.
2. Z-score normalise each column: $z = \frac{x - \mu}{\sigma}$. If $\sigma = 0$, use $\sigma = 10^{-6}$ to avoid division by zero
3. Compute $P^* = \sum w_{k,i} \cdot Z_{k,i}$ (weighted centroid over all criteria), where $\sum w_i = 1$
4. Min-max scale to $[0, 100]$. If all P^* values are identical, return 50 for all.

The endpoints 0 and 100 thus represent the relative minimum and maximum within the alternative set, with the zero point being purely relational rather than an absolute preference origin.

Within Preferendus' IMAP method, preference-performance optimisation — the search for a best-fit — is operationalised through an Inter-Generational Solver (IGS), in which population-based metaheuristic search processes serve as exploration mechanisms within the constrained search space. In particular, genetic algorithms (GAs) and swarm intelligence methods belong to the broader class of population-based metaheuristics that provide alternative instantiations of the search component. More generally, the IGS architecture decouples stochastic search from preference-based decision aggregation by embedding interchangeable population-based metaheuristics a higher-level preference-consistent aggregation layer.

Survival Selection

IMAPSurvival: The two-pass survival operator that replaces NSGA-II's RankAndCrowdingSurvival.

Two-pass aggregation:

- First pass: runs IMAP over the full pool (parents + offspring) to identify and discard low-quality candidates (score ≤ 40).
- Second pass: runs IMAP over the surviving candidates only, producing the final scores used for selection.

Discard phase / threshold (score ≤ 40): After the first pass, any individual scoring ≤ 40 is removed from the candidate pool before the second pass. Skipped if it would leave fewer than N survivors. Under Constraint dominance principle (CDP), feasible individuals are always protected from discard when there are fewer feasible solutions than N .

Candidates: The pool that remains after the discard phase. The second-pass IMAP scores are computed on this set.

Truncation selection: Deterministically keeps the top N candidates ranked by final IMAP score. Under CDP the sort is lexicographic: feasible individuals rank above infeasible ones, then by IMAP score within each group.

Tournament selection: Runs N binary tournaments on the candidate population. Each tournament samples two candidates at random. The higher-scoring (and feasibility-preferred under CDP) candidate wins. Preserves more genetic diversity than truncation at the cost of weaker selection pressure.

Constraint Handling

Constraint Dominance Principle (CDP): A feasible individual always beats an infeasible one in any comparison (survival or tournament), regardless of IMAP score. Applies whenever there are constraints.

Constraint Violation (CV): Total constraint violation of an individual: $CV = \sum \max(0, g_j)$ over all constraints. A solution is feasible if and only if $CV = 0$.

CV preference function: An explicit preference applied to each individual's total CV. When set, all stakeholder weights are scaled by $(1 - cv_weight)$ and the CV criterion receives weight cv_weight .

When the population is fully feasible, all CV scores ≈ 100 and $\text{std} \approx 0$, so the CV term contributes essentially nothing.

CV weight: Fraction of total aggregation weight assigned to the CV criterion. Must be in (0, 1) when `cv_pref_fn` is set.

Elitism and Best Tracking

Current best: The highest-scoring feasible individual seen across all completed survival passes. Injected into the pool at every generation to update its preference score with the new population. Updated only when a strictly better, genuinely different solution is found in the same generation's second-pass scoring context.

Current_best_gen: The generation index at which `current_best` was last updated.

Hall of fame: Ordered list of all outgoing `current_best` values. Every time `current_best` is replaced, the old one is appended here. Provides a history of all best-found solutions across the run. Note: general concept in GA's.

Elitism guarantee: Before each generation's selection call, `current_best` is injected into the population if not already present. After selection, `current_best` is force-kept in the survivors if it would otherwise be dropped.

Archive

Memory structure (best archive): Optional storing of previously best individuals. When enabled, archive members are appended to the candidate pool's objective-value dict before the second-pass IMAP scoring. This extends the normalisation reference frame across generations, stabilising Z-score computation.

Archive stabilisation: Archive members participate in Z-score normalisation only; they are never returned as survivors. Including them prevents the score distribution from collapsing when the live candidate pool becomes too homogeneous.

Archive max size: Maximum archive size. When exceeded, we score all archive members against each other and discards the lower-scoring 50%, retaining at least one individual.

Inherited NSGA-II Infrastructure

Population size: Number of individuals maintained in the population across generations.

Population: Container for a set of Individual objects. Carries evaluated objective values (F), constraint values (G), decision variables (X), and any custom attributes (here: `imap_score`, `is_feasible`).

Individual: Single solution with fields X (decision variables), F (objectives), G (constraints), and arbitrary named attributes.

Crossover / Mutation: Standard pymoo operators inherited from NSGA2/GeneticAlgorithm; IMAP-GA does not modify them.

References

[1] Wolfert, A.R.M. et al. (2023). *Open Design Systems (ODESYS)*. TU Delft. Repository: <https://github.com/TUdelft-Odesys>

[2] Özer, E. and Wolfert, A.R.M. (2026). Ordering Is Not Deciding: Towards Preference-Consistent Design--Decision Optimisation. *To be published*.

[3] Timp, L.J.K., Yorke-Smith, N. & Wolfert, A.R.M (2026). Solving highly constrained multi-objective decision problems with preference-guided metaheuristics: The IMAP-IGS framework. *To be published*.

[4] Wolfert, A.R.M. (2026). Unique preference aggregation in design and decision making. *arXiv*. <https://doi.org/10.48550/arXiv.2601.19759>

[5] Wolfert, A.R.M. (2026). Preference-Based Optimisation for Integrated Design and Group Decision-Making. *arXiv preprint arXiv:2603.19050*. Available at: <https://arxiv.org/abs/2603.19050>. Preferendus software: IMAP-IGS

